Application of the Rjindael Field to Hashing

Kieran Harvie

Copyright ©May 11, 2023. All Rights Reserved.

Contents

1 Introduction **3**
- 1.1 Structure of the Repository . . . 3
- 1.2 Hashing 101 . . . 3
  - 1.2.1 Hashing Terminology . . . 4
- 1.3 Abstract Algebra 101 . . . 5
- 1.4 Field Theory 101 . . . 5
  - 1.4.1 Constructing Fields . . . 7
  - 1.4.2 Finite Fields . . . 9
- 1.5 The Rijndael Field . . . 10
  - 1.5.1 Generators . . . 12
  - 1.5.2 Notation . . . 12

2 Matrix Methods **13**
- 2.1 Gaussian Elimination . . . 13
  - 2.1.1 (pointlessly) Abstract Algebra Preamble . . . 13
  - 2.1.2 (useful) Matrix Preamble . . . 13
  - 2.1.3 The Algorithm . . . 15
- 2.2 Application to Hashing . . . 15

3 Polynomial Methods **16**
- 3.1 Interpolation . . . 16
- 3.2 Evaluation . . . 16
  - 3.2.1 Horner's scheme . . . 16
  - 3.2.2 Estrin's scheme . . . 16
  - 3.2.3 Parallel evaluation . . . 16
- 3.3 Application to Hashing . . . 16

4 Advanced Encryption Standard **17**

A Appendix **18**
A.1 Adjoining a Number to a Field 18
A.2 Generator Table 19
A.3 Side-Channel 19
A.4 Matrix Arithmetic 20
A.4.1 Diagonalization 20
A.5 Polynomial Arithmetic 22

Introduction

General introduction This is an accompanying document to the repository.

It is meant to approachable to someone that has freshman understand of math or programing, while possibly requiring some external research to use to the fullest. And hopefully being giving enough of a taste to inspire that research.

And maybe even an ambassador for abstract algebra and cryptography for people that are interested in those things but need practical applications to make them stick.

There is a bit of a hike to get towards the Rijndael Field and people familiar with abstract algebra can skip the rest of this introduction.

1.1 Structure of the Repository

1.2 Hashing 101

For our purposes a hash function is an any function that inputs a string and outputs a number. The most direct example is converting the first character to it alphabetical number (A=1, B=2, etc.):

$$\text{hash(Apple)} = 1, \quad \text{hash(Orange)} = 15$$

There are two main applications for hashing: The first is performance, as operations on strings may be more expensive then on numbers so an efficient and well designed hash may allow us to work better on the numbers instead. The second is cryptography, that is the practice of communicating secrets, where we want to hash the string and communicate the number in hopes of keeping the string secrete.

There is a clear tension between these two applications. For performance you'd want as much information about the initial string easily assessable in the hash. But for cryptography you'd want as much information hidden as possible while still allowing communication.

This document aims to serve as an introduction, so wont optimize for either application, but I bring them up this early to wet the readers appetite about why they might care about hashing. **But also to state that the reader should not apply any algorithm here for cryptographic purposes without consulting a professional first.** At least one technique used makes the algorithm susceptible to a side-channel attack, and even if I avoided that technique there would be another one I'm ignorant of.

1.2.1 Hashing Terminology

With applications and warning asides there are some more things the reader should know. First is that in common use the term hash can mean both the hash function and the resulting number from that function.

Secondly there are four general terms worth knowing:

Collision: A collision is when two strings have the same hash. This is broadly undesirable since any operation we want to preform on the hash has to work for both strings.

Hash Table: A common application of hash functions in environments where accessing a look-up table with string keys are inefficient compared to number keys. We basically make a look-up table with number keys and when asked to look-up a string key we hash it first and use the result as the integer key. This will require us to deal with collisions as they come up.

Perfect Hash Function: A perfect hash for a given set of strings is hash function where none of strings collide.

Minimal Perfect Hash Function: A minimal perfect hash for a given set is a perfect hash function where all outputs of the string are less then or equal to the size of the set. In terms of features, but not necessarily performance, this is the ideal hash function as there are no collisions to deal with and no wasted numbers taking up space.

1.3 Abstract Algebra 101

A summary of Abstract Algebra two sentences would be something like:

> Abstract Algebra is the study of Algebraic Structures.
>
> Algebraic Structures are a set of objects (e.g. Numbers) and operations (e.g. $+, \times, -, \sqrt{\ }$) that follow certain axioms (e.g. $a + b = b + a$).

One thing I want to stress is that from the abstract algebra viewpoint it is taken as an axiom that $a + b = b + a$.

While one could make arguments about why $1 + 2 = 2 + 1$ in abstract algebra it's just kind of assumed as an axiom in algebraic structures of numbers. And we instead focus figuring out what are the logical conclusions of all the assumed axioms.

The reason this is a useful thing to do is that it enables us reuse arguments for other structures given they have the same axioms used in the argument. Consider a proof of the difference of two squares:

$$\begin{aligned}(a-b)(a+b) =& a(a+b) - b(a+b) \\ =& (a^2 + ba) + ((-b)a + (-b)b) \\ =& a^2 + (ba + ((-b)a) + -b^2 \\ =& a^2 + a(b + (-b)) - b^2 \\ =& a^2 + a \times 0 - b^2 \\ =& a^2 - b^2\end{aligned}$$

We can list all the axioms used[1] and apply know that the difference of two squares is the natural conclusion of these axioms. And hence apply this formula to any algebraic structure that follows the axioms used.

1.4 Field Theory 101

> Fields are Algebraic structures that act like the rational numbers $\mathbb{Q}$ (i.e. Fractions with $+, -, \times, \div$)

[1] e.g. $a(b+c) = ab+ac$, $ab = ba$, $a+b = b+a$, $0 \times a = 0$, $b+(-b) = 0$, $(-b)b = -b^2$, ...

There are a lot of axioms for a field, and I don't think listing them all here would be useful. Especially when the reader can get the gist with an example.

Consider multiplication and addition of integers less then 5 where if the answer is greater than 5 we divide it by 5 and take the remainder:

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Here are some interesting properties of these tables:

- They are mirrored around the diagonal. Meaning $a + b = b + a$ and $ab = ba$.
- The number 0 has the properties $0 + a = a$ and $0 \times a = 0$.
- The number 1 has the property $1a = a$.
- For every number a there is a unique number b such that $a + b = 0$. (Meaning we can define $-a = b$).
- For every nonzero number a there is a unique number b such that $ab = 1$. (Meaning we can define $\frac{1}{a} = b$).

These fact combined means we can define and manipulate fractions in this structure the same way we would in $\mathbb{Q}$. (Using $\frac{a}{b} = a \times \frac{1}{b}$ as needed). These operations (Addition, Multiplication, Negation, and Reciprocation) are called the field operations and the fact we can do all of them makes "Integers less then 5 where we can take the remainder of any operation"[2] a field.

I said one of the benefits of abstract algebra is the reuse of arguments where the axioms still apply. So lets test this by applying the difference of squares proof from the previous section to 2 and 4:

$$4^2 - 2^2 = 1 - 4 = -3 = 2$$

[2]For reasons that will become clear later this structure is called $GF(5)$

and

$$(4-2)(4+2) = 2 \times 1 = 2$$

Which are equal, as expected.

It's also worth noting what we can't do in this structure:

- Square root: Notice that in the $\times$ table diagonal there is no 2 or 3 this means we can't meaningfully talk about $\sqrt{2}$ or $\sqrt{3}$.
- Order: Notice that $4 + 1 = 0$ but that normally we would say $0 < 4$ however adding 1 to both sides gives $1 < 0$.

That's fine because neither of these concepts are part of the field axioms and this neutrality is what makes abstract algebra so powerful.

Despite not being part of the field axioms, order makes sense in $\mathbb{Q}$, and square root makes sense for positive numbers in $\mathbb{R}$. This is because axioms in abstract algebra only guarantee the structure they are meant to describe and don't prohibit additional structure. You can pick and choose axiom and how they interact and take them to their logical conclusions while not having to repeat work what only requires certain subset of the axioms.

We already implicitly took advantage of this power at the start when we defined field as like $\mathbb{Q}$ but not $\mathbb{R}$. Operations like square roots working for all positive integers aren't going to be important for our hashing application so we are using a field to work because that's all we need.

1.4.1 Constructing Fields

To really use the full potency of Field Theory, and eventually use the Rijndael field, we need to move past describing fields that we already know and move to constructing new fields.

The most obvious way to construct a field is to take an existing field F and add another number a to it while doing whatever is necessary to make sure the field axioms still apply. This process is called the adjoining of a to F.

As observed earlier there is no $\sqrt{3}$ in the field F of integers remainder 5.

What if we define a new set G such that:

$$G = \left\{ a + b\sqrt{3} \,|\, a, b \in [0, 4] \right\}$$

With addition, multiplication, and negation working as expected (taking the remainder as needed):

$$\begin{aligned}
(a_0 + b_0\sqrt{3}) + (a_1 + b_1\sqrt{3}) =& (a_0 + a_1) + (b_0 + b_1)\sqrt{3} \\
(a_0 + b_0\sqrt{3}) \times (a_1 + b_1\sqrt{3}) =& (a_0b_0 + 3a_1b_1) + (a_0b_1 + a_1b_0)\sqrt{3} \\
-(a_0 + b_0\sqrt{3}) =& (-a_0) + (-b_0)\sqrt{3})
\end{aligned}$$

And reciprocation defined as:

$$\frac{1}{a + b\sqrt{3}} = \frac{1}{a^2 - 3b^2}(a - b\sqrt{3})$$

Hence we have created a new field G that contains F as a subfield, when $b = 0$ and includes a square root of 3 while satisfying of the field axioms.

The theory for constructing new fields was laid by groundbreaking work 19th mathematician Évariste Galois. Like much groundbreaking work before and after it wasn't understood in it's time. With influential mathematicians like Cauchy rejecting Galois' papers and Poisson calling it "incomprehensible". This lack of understanding was not helped when Galois' political activism landed him in jail after the French Revolution of 1850 and upon being release immediately entering a duel and dying.

Surly such a groundbreaking idea from such a interesting life must be too complex for regular people to understand? Well what this groundbreaking work focused on was finding the roots of polynomials, something you likely did in high school.

Because multiplication and addition are field operations given a field F we can define the set of polynomials with coefficients in F as $F[X]$. And if a can be written as the solution to a polynomial in $F[X]$ we can create a new field $F(a)$ where the elements of the new field are polynomials in the old field evaluated at a, this set is denoted $F[a]$.

This process is the same as the one shown in the beginning of the section where the polynomial used there was $X^2 - 2$

As abstract as this all seems you have likely done this implicitly before if you've ever used complex numbers. You were likely told to treat $\mathbb{C}$ and $\mathbb{R}$ but with an extra number i such that $i^2 = -1$. You can now see that $\mathbb{C}$ and $\mathbb{R}$ are fields and that $\mathbb{C}$ is what you get when you follow this adjoining process on $\mathbb{R}$ with $X^2 + 1$.

1.4.2 Finite Fields

One final piece of theory before the Rijndael Field is about the special case where a field as has finite number of elements.

These finite fields are also called Galois Fields, in honor of Galois, and a finite field of size n is denoted $\mathrm{GF}(n)$ or $\mathbb{F}_n$. The field of "Numbers less than 5 where we take the remainder after operations" that we have used had the proper name is $\mathrm{GF}(5)$ or $\mathbb{F}_5$ and the and when adjoined with $\sqrt{3}$ it would become $\mathrm{GF}(5)(\sqrt{3})$ or $\mathbb{F}_5(\sqrt{3})$.

There is a lot of interesting facts we know about finite fields, but perhaps a better word would be restrictions.

Eagle eye readers may have already figured out one of these restrictions from the previous notation. How can $\mathrm{GF}(5)$ distinguish between two different finite fields of size 5? Well they don't have to because all finite fields of the same size are "isomorphic" with each other. Which is an abstract algebra way of saying you can change what you label everything ($1 \to a$, $2 \to \triangle$...) but the structure is the same.

Those multiplication tables where the only possible multiplication tables for five elements that are also a field. And since all the structure came from those tables it doesn't matter what we call the elements.

I don't know of any easy proof/argument for this so I'm going to have to leave it as out of scope, but readers are encouraged to do their own research.

This means that $\mathrm{GF}(5)(\sqrt{3}) = \mathrm{GF}(25)$, which cleans up the ugly notation on the right hand side.

Another interesting restriction on structure is that not all n can have a finite group. Only n that are the power of some prime p can have a finite group.

The intuition as to why prime numbers might be important consider the constructing a group of size pq, where p and q are different primes, like we constructed the field of size 5.

For this set to be a field there needs to be a reciprocal of p, meaning an x such that px has remainder 1 when divided by pq. Now we can assume $x < q$ because if $x > q$ we take $x - q$ as the new x since $p(x - q) = px - pq$ which has the same remainder when divided by pq and we can repeat this step as much as we need until $x < q$. But if $x < q$ then $px < pq$ meaning we don't have to take the remainder when calculating px meaning $px = 1$, no extra step needed. But $p > 1$ and $x \geq 1$ meaning $px \geq 1$ which contradicts $px = 1$, hence there can be no reciprocal of p in a field of size pq constructed this way.

This argument actually only shows we can't construct a field of size pq the way we constructed 5, but the gist that there is a problem with reciprocals when we mix primes remains.

While not useful to for out treatment of the Rijndael Field there is one more cool property of finite fields. That for the field $\mathrm{GF}(p^n)$ we have:

$$(a + b)^p = a^p + b^p$$

No binomial expansion needed.

1.5 The Rijndael Field

The Rijndael Field is another name for $\mathrm{GF}(2^8)$ and first entices the math savvy programmer by it's size. 2^8 is the same as the number of states in a byte, and there are a tone of useful algorithms for $\mathbb{R}$ and $\mathbb{Q}$ that only rely on field operations. So can these algorithms become useful if applied to bytes through the Rijndael Field?

Well the best answer is to check, and the best way to check to to construct $\mathrm{GF}(2^8)$.

Lets start with the multiplication and addition tables for GF(2):

+	0	1
0	0	1
1	1	0

×	0	1
0	0	0
1	0	1

Clearly $+$ is the XOR operator and $\times$ is the AND, already promising for computational applications.

Now I present, without explanation, the following element of $\mathrm{GF}(2)[X]$

$$p(X) = X^8 + X^4 + X^3 + X + 1$$

And assert that it constructs $\mathrm{GF}(2^8)$ when used in the same way as in the construction section of Field Theory 101. This can be proved rigorously with Galois Theory but is outside the scope of this document. And for our purposes all that matters is that it does work and generates a representation of $\mathrm{GF}(2^8)$:

$$\mathrm{GF}(2^8) = \{a_0 + a_1X + a_2X^2 + a_3X^3 + a_4X^4 + a_5X^5 + a_6X^6 + a_7X^7 \mid a_n \in [0,1]\}$$

Addition and multiplication follow normal polynomial arithmetic, remembering that the coefficients are in $\mathrm{GF}(2^8)$:

$$\begin{aligned}
(1 + X^2 + X^5 + X^7) + (X^3 + X^5) =& 1 + X^2 + X^3 + (1+1)X^5 + X^7 \\
=& 1 + X^2 + X^3 + X^7 \\
(X^2 + X^3)(1 + X^5) =& 1 + X^2 + X^3 + X^5 + X^8 \\
=& 1 + X^2 + X^3 + X^5 + (p(X) - X^4 - X^3 - X - 1) \\
=& -X + X^2 - X^4 + X^5 + p(X) \\
=& X + X^2 + X^4 + X^5 + p(X) \\
=& X + X^2 + X^4 + X^5
\end{aligned}$$

(Note that the last line comes from how $\mathrm{GF}(2^8)$ was constructed through $p(X)$ means $p(X) = 0$).

This is very promising as we can convert an element of $\mathrm{GF}(2^8)$ to a byte by reading off coefficients from a 7th degree polynomial. Addition in $\mathrm{GF}(2^8)$ is just bitwise XOR. And Multiplication is a convolution where we trim of element greater than X^8 by using $p(X) = 0$.

Multiplication is already a computationally tractable but there is a better method using "Generators". So with simple addition, easy conversion between $\mathrm{GF}(2^8)$ and bytes, and the promise of a better multiplication I hope you are convinced that the Rijndael Field is computationally promising.

1.5.1 Generators

A generator for a field is an element g such that for any non-zero element a there is some integer n such that:

$$a = g^n$$

Where exponentiation is just iterative multiplication, nothing special.

1.5.2 Notation

Carrying X around is annoying and we already have notation for bytes, so

Matrix Methods

From an abstract algebra a matrix is actually a type of tensor as field[it the underlying object is just a ring it's called a module. Say how walking through a 3×3 examples is useful to show only field operations is used. set spell not working?

MAKE SURE TO USE THE APPENDIX!!

2.1 Gaussian Elimination

2.1.1 (pointlessly) Abstract Algebra Preamble

From an abstract algebra viewpoint, the choice of dual space basis is arbitrary provided they have the right span. We can get such a new dual basis $\mathbf{e}'_{\mathbf{i}}$ from an old basis $\mathbf{e}_{\mathbf{j}}$, provided T is non-singular, through:

$$\mathbf{e}'_{\mathbf{i}} = T(\{\mathbf{e}_{\mathbf{j}}\})$$

So we can transform relationship between the coefficients of tensors in one basis it to another, and if T is affine then the new coefficients are directly transformed through T.

Three particular choices of T are swapping two the basis elements, multiplying a basis element by a non-zero constant, and adding a multiple of basis element index to another.

2.1.2 (useful) Matrix Preamble

Abstract tensor analysis is cool, for the type of people that like that type of thing, but the useful result of the previous subsection is that for matrices a,

b, and M, plus an invertible matrix T we have:

$$M_i^j a^i = b^i \Leftrightarrow T_k^j M_i^k a^i = T_k^j b^k$$

(And this result can be verified directly by multiplying by T or T^{-1} and short-circuiting the previous preamble).

Bellow are examples of the three previous particualr choices of T, hence forth referred to as elementary row operations, for 3×3 matrices:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{31} & m_{32} & m_{33} \\ m_{21} & m_{22} & m_{23} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ am_{21} & am_{22} & am_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & a & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} + am_{21} & m_{32} + am_{22} & m_{33} + am_{23} \end{bmatrix}$$

Since these elementary row operations can be represented through matrices their composition can be represented through matrix multiplication. Hence we can build more complex row operations from multiplying a chain of elementary row operations together.

For example say we want a transform T that takes the a 3×3 matrix and makes it first column $[1, 0, 0]$. Assuming $m_{1n} \neq 0$, we have:

$$\begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 0 & m_{22}m_{12}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{12}^{-1} - m_{13}m_{11}^{-1} \\ 0 & m_{32}m_{32}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{31}^{-1} - m_{13}m_{11}^{-1} \end{bmatrix}$$

Hence:

$$T = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix}$$

Gaussian Elimination is simply using this process to find a matrix T such that $T(M) = I$ by interatvly zeroing the outer, non-diagonal, elments:

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & m'_{22} & m'_{23} \\ 0 & m'_{32} & m'_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & m''_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This calculates M^{-1} as the product of matrices representing elemetry row operations. Note that this always works when M is invertable, adafsd but we may need to swap row such that errent zeros put directly into their destination to avoid dividing by zero.

Rrwrite to bring in the opening. Jordan normal form??

2.1.3 The Algorithm

Matrices can easliy be represented as 2d-arrays, elemetry row operations aren't hard to implement either, and the iterative nature of finall equation directly suggests the structure of the algorith. However there is one more observation instead of keeping in memory all the elementry row operation, just directly apply them to the B as we go. augmented matrices are exceleent at facilitating this.

2.2 Application to Hashing

liner and human names $Ma_0 = Ma_1$ for invertable M preprocesing/ offsetting

Polynomial Methods

3.1 Interpolation

Lagrange

3.2 Evaluation

3.2.1 Horner's scheme

$$y_0 = a_n$$
$$y_k = a_{n-k} + xy_{k-1}$$

3.2.2 Estrin's scheme

3.2.3 Parallel evaluation

3.3 Application to Hashing

Minimal perfect hashing.

Advanced Encryption Standard

As alluded to in the introduction one use of the Rijndael field is the Advanced Encryption Standard.

The basis of AES is the Rijndael field and some matrix operations, meaning we inadvertently have all tools necessary to give an overview. Not sure I will implment this in C.

Appendix

A.1 Adjoining a Number to a Field

TODO: clean up and probably add an appendix for Bézout's identity and the notation $F[X]$. When adjoining a number a to field F there are two steps:

1. Making a set which includes all the elements of F as well as a.
2. Making sure the all the field axioms still apply to the new set.

While it easy to make sure addition, subtraction, and multiplication work by defining a new set which is the sum of powers of a times elements in F:

$$\left\{ \sum_n f_n a^n \mid f_n \in F \right\}$$

But what about reciprocation?

Well for this discussion we have two similar but distinct notations:

1. $F[a]$ is the ring[1] with the set defined as above.
2. $F(a)$ the minimal field that includes the element a and the subfield F.

These objects aren't necessarily the same. Consider a number a that isn't the solution of some polynomial in F. (I state, without proof, that this is the case $a = \pi$ and $F = \mathbb{Q}$). Then $F(a) \neq F[a]$ because $a^{-1} \notin F$.

Now consider the case where a is the solution of some polynomial over F.[2] Without lost of generality let f be the minimal polynomial such that

[1] An algebraic structure that is a field without reciprocation

[2] This property is called a being algebraic over F

$f(a) = 0$. Let g be a different polynomial in $F[X]$ By the minimality of f either $g(a) = 0$ or f and g are relatively prime. If $g(a) = 0$ then $g = 0$ in $F[a]$ meaning it doesn't need a reciprocal, hence f and g are relatively prime, and hence from Bézout's identity we have polynomial r,s in $F[X]$ such that:

$$f(x)r(x) + g(x)s(x) = 1$$

But when we substitute in a we have:

$$g(a)s(a) = 1$$

Hence g has a reciprocal in $F[a]$ and hence $F[a] = F(a)$

In practice a lot of numbers adjoined to a field are defined as roots of polynomials it is often the case that $F[a] = F(a)$.

A.2 Generator Table

A.3 Side-Channel

I've said in a few places that using generators to simplify field arithmetic makes the implementation susceptible to side-channel attacks. But how? And what is a side-channel attack?

A side-channel is something about the implementation of an algorithm or protocol that lets an attacker get information, rather than something inherently wrong with the algorithm or protocol itself.

The easiest to explain side-channel is the timing. Consider a guarded door and to get access you have to tell the guard a password. You tell the guard two different incorrect passwords but the guard rejects one immediately but waits till you're half way through for the second one. You might reasonably conclude that first password is wholly wrong but that the first half of the second one is correct. By doing this enough times and keeping track of the time the guard rejects the password you can probably figure out the password. You guessing the password has nothing to do with the cryptography of a password but the implementation by the guard, instead they should have waited till you've said the whole password before rejecting

it, and hence is a side-channel attack.

Using a look-up table to compute field operations opens you up to a caching side-channel attack.

Because the look-up table is to large to be stored in the CPU registers it will be accessed from memory. Accessing the table from memory means it's likely to be cashed. The table being cached means someone with the ability to monitor cache access can see what field elements you're using, and maybe work backwards to get some of the string key that was hashed. Hence leaking the string.

This makes using look-up tables unsuitable for application where the string keys are sensitive information.

Before closing this topic I want to bring up that side-channels can include the physical properties of computers as well. Consider a naïve attempt at stopping a timing attack where the guard waits to reject the password but is still noticeably paying less attention. An attacker can still guess the timing from when the guard stops paying attention. Similarly idling a computer till a set amount of time passes for all password validation attempts might cause a change in power consumption or a noticeable reduction in cooling. Hence even power-draw or acoustic properties of a computer are possible side-channels for attacks.

A.4 Matrix Arithmetic

A.4.1 Diagonalization

The description of Gaussian Elimination and the claim that it works with any invertible matrix may make some readers that half remember a linear algebra course nervous. This is because process described is similar to another process, diagonalization, which is not guaranteed to work just because the input matrix is invertible.

A diagonal matrix is one where all non-diagonal elements are zero and

transcribed as Diag:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 7 \end{bmatrix} = \mathrm{Diag}(1, 3, 2, 7)$$

Such matrices are important they are the same as the matrix for the elementary row operation of scaling the rows, making matrix multiplication invoking them conceptually and computationally easy. For example:

$$\mathrm{Diag}(a_0, a_1, \ldots a_n)\,\mathrm{Diag}(b_0, b_1, \ldots b_n) = \mathrm{Diag}(a_0 b_0, a_1 b_1, \ldots a_n b_n)$$

Diagonalization is the process of taking a matrix M and finding a diagonal matrix D and invertible matrix P such that: [3]

$$MP = PD$$

And this process is important because it enables the simplification of calculations by canceling P's and utilizing diagonal matrix multiplication:

$$\begin{aligned} M^3 =& (PDP^{-1})^3 \\ =& (PDP^{-1})(PDP^{-1})(PDP^{-1}) \\ =& PD(P^{-1}P)D(P^{-1}P)DP^{-1} \\ =& PD^3P^{-1} \\ =& P\,\mathrm{Diag}(a_k)^3 P^{-1} \\ =& P\,\mathrm{Diag}(a_k^3) P^{-1} \end{aligned}$$

But unfortunately diagonalization isn't guaranteed for invertible matrices. The conditions of when it works involves something called eigenvalues, which are outside the scope of this document. But what is concerning is how similar this looks to Gaussian Elimination, which given $MA = B$ involves finding an invertible P such that:

$$M'A = PMA = PB$$

[3] Often equivalently written as $M = PDP^{-1}$

Where M' is M after some elementary row operations. What is important to notice is that in diagonalization P is on the right side of M and the left side of D. But during Gaussian Elimination P is on the left side of both M and B. Since matrix multiplication isn't commutative this makes the processes very different and lays nerves to rest.

Readers interesting in learning more about diagonalization and it's uses are encouraged to first loop up eigenvalues and then "The Jordan Normal Form".

A.5 Polynomial Arithmetic